Workload-Aware Adaptive Page Replacement

Name:Arpit Raj

Reg.No:24BCE2897

Implementation Details

1. Project Overview

Workload-Aware Adaptive Page Replacement is an Operating Systems project that demonstrates how page replacement algorithms manage memory when the number of pages requested by a process exceeds the available physical memory frames. When a required page is not present in memory, a page fault occurs, and the operating system must decide which existing page to replace.

This project implements and compares four page replacement algorithms: First-In, First-Out (FIFO), Least Recently Used (LRU), Optimal, and Adaptive Page Replacement. It includes a C++ simulator and a standalone web-based simulator that displays algorithm performance through tables, graphs, and step-by-step execution traces.

2. Objectives

- To understand and implement different page replacement algorithms.
- To compare algorithms based on page faults, page hits, and hit and fault ratios.
- To develop an adaptive strategy that selects between FIFO and LRU using previously observed page-reference behaviour.
- To provide a user-friendly website for simulating and visualizing page replacement.
- To test the algorithms using different page-reference sequences and memory frame counts.
- To upload the complete implementation to GitHub for review and submission.

3. Technology Stack and Configuration

The project uses the following technologies:

- C++: Used to implement the command-line page replacement simulator.
- HTML: Defines the structure and components of the website.
- CSS: Provides the website layout, dark theme, colors, tables, and responsive styling.
- JavaScript: Implements the website simulations, calculates performance metrics, and updates the results and graphs.
- Web browser: Used to run the website locally.
- GitHub: Used to store and share the project source code and documentation.

The website consists of three files: index.html, style.css, and script.js. It can be run locally by opening index.html in a web browser. The website does not require a database, login system, or additional framework.

The C++ simulator requires a compatible C++ compiler. The exact compiler version and compilation command should be documented according to the development environment used.

4. Algorithms Implemented

FIFO (First-In, First-Out): Replaces the page that has been in memory for the longest time. It is simple to implement but may replace a page that will soon be needed again.

LRU (Least Recently Used): Replaces the page that has not been accessed for the longest time. It uses recent access history to make replacement decisions.

Optimal Page Replacement: Replaces the page whose next use is farthest in the future. It is primarily used as a theoretical benchmark because it requires knowledge of future page references.

Adaptive Page Replacement: The website uses two hidden shadow memories to track FIFO and LRU performance over a recent window of up to eight references, including the current reference. When a replacement is needed, it selects the strategy with fewer faults in that window. If both strategies have equal faults, FIFO is selected. This approach uses observed references rather

than future information. Adaptive is a heuristic and is not guaranteed to outperform FIFO or LRU on every workload.

The C++ and website implementations should be documented separately if their Adaptive decision logic differs.

5. Website Features and Implementation

The website provides a simple dashboard for comparing all four algorithms using the same page-reference sequence and number of frames.

Its main features include:

- Input fields for the page-reference sequence and number of frames.
- A simulation button to execute the algorithms.
- A results table displaying page faults, page hits, hit ratio, and fault ratio.
- A bar chart comparing page faults across algorithms.
- A step-by-step simulation table showing frame contents and whether each reference results in a hit or fault.
- An Adaptive decision table showing the selected strategy and replacement information.

These features make it easier to observe how different replacement strategies behave under different workloads.

6. Testing and Validation

The simulator should be tested with sequential, repeating, textbook, and mixed page-reference sequences. Tests should use different numbers of memory frames to observe changes in algorithm performance.

The following checks should be performed:

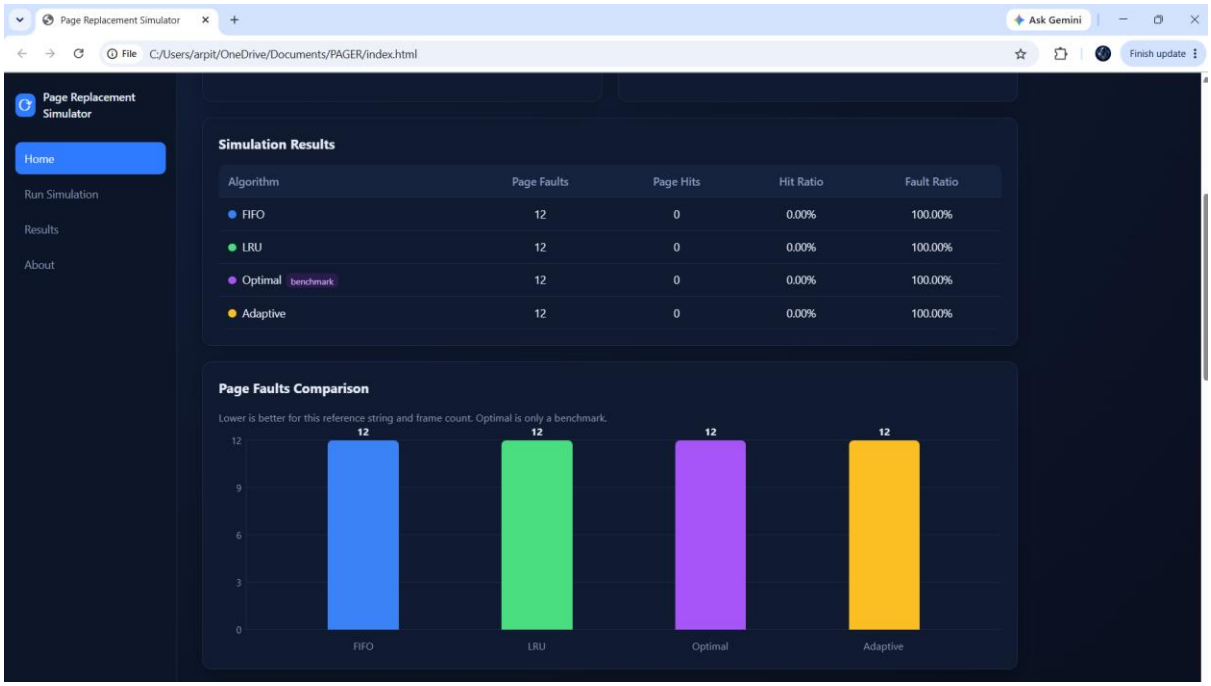
1. The number of page hits plus page faults must equal the total number of page references.
2. The hit ratio and fault ratio must add up to 1, or 100% when expressed as percentages.
3. The results table and bar chart must update when new inputs are entered.

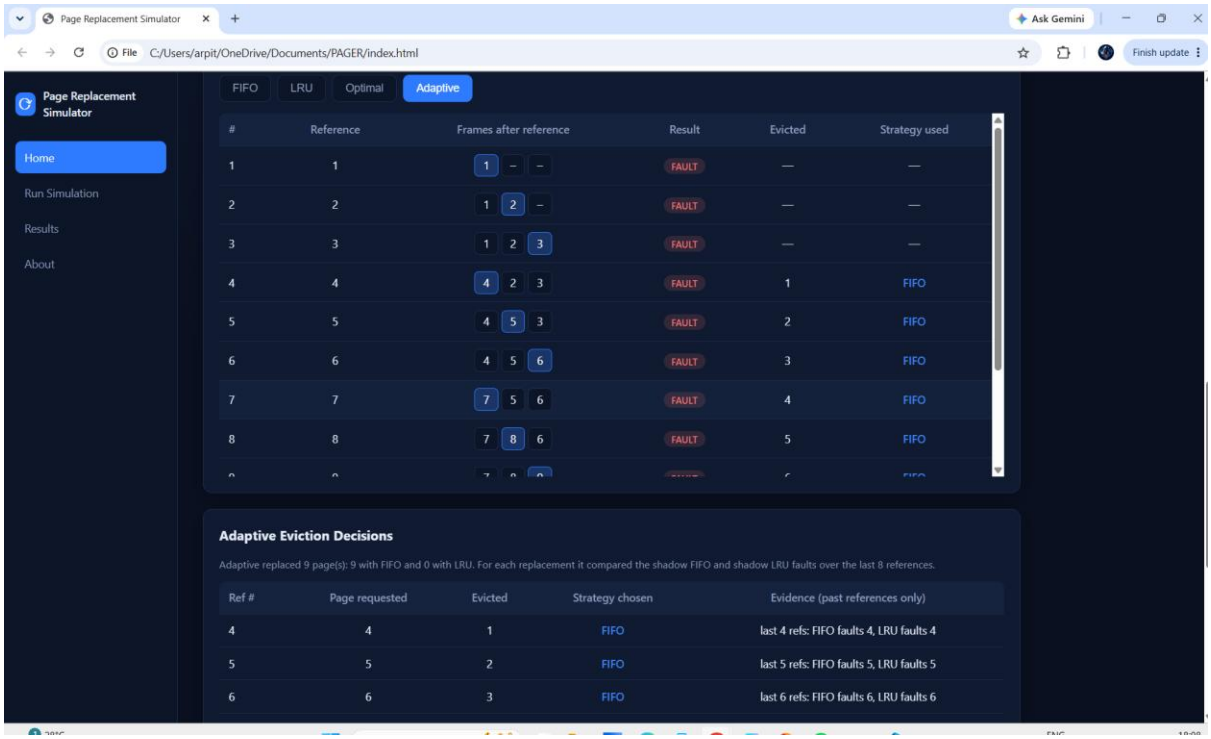
4. The step-by-step trace must show the correct frame contents and hit or fault status.
5. Adaptive decisions must be displayed when replacements occur.
6. Optimal should produce no more page faults than another correctly implemented algorithm for the same reference sequence and frame count.

Screenshots of the actual website, results table, graph, and step-by-step simulation should be included as evidence of testing.

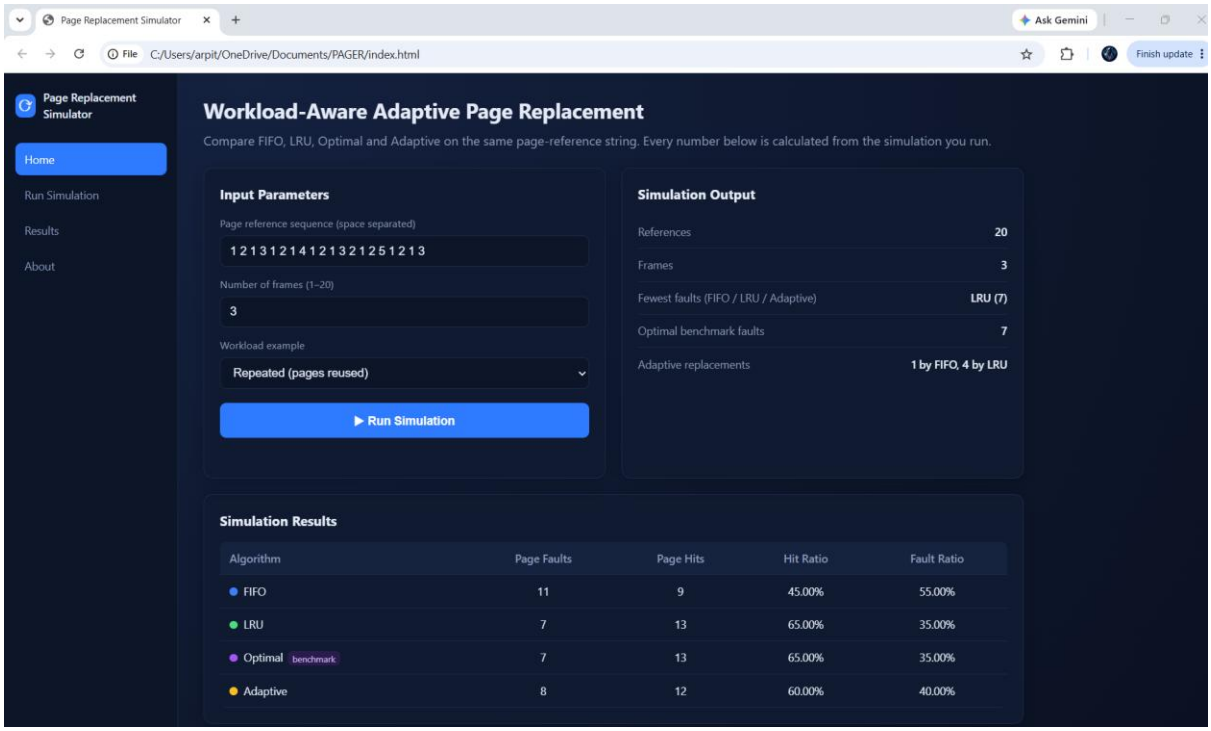
7. Result and Screenshots

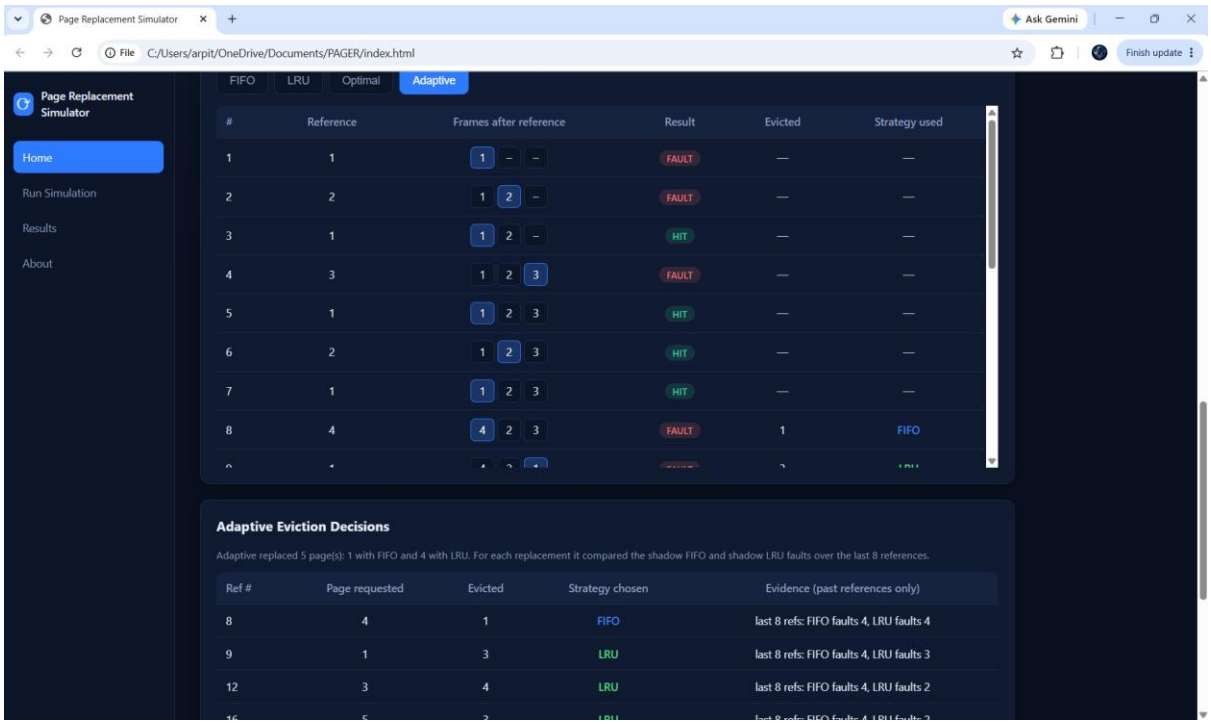
Sequential Reference:



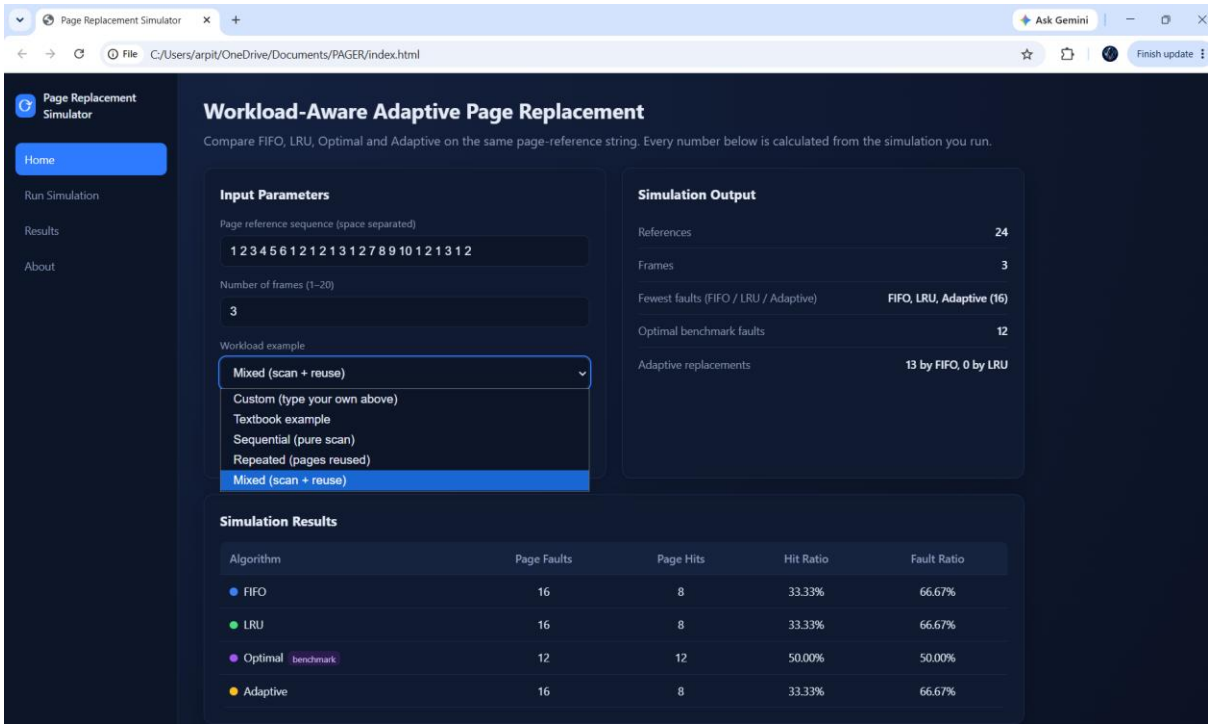


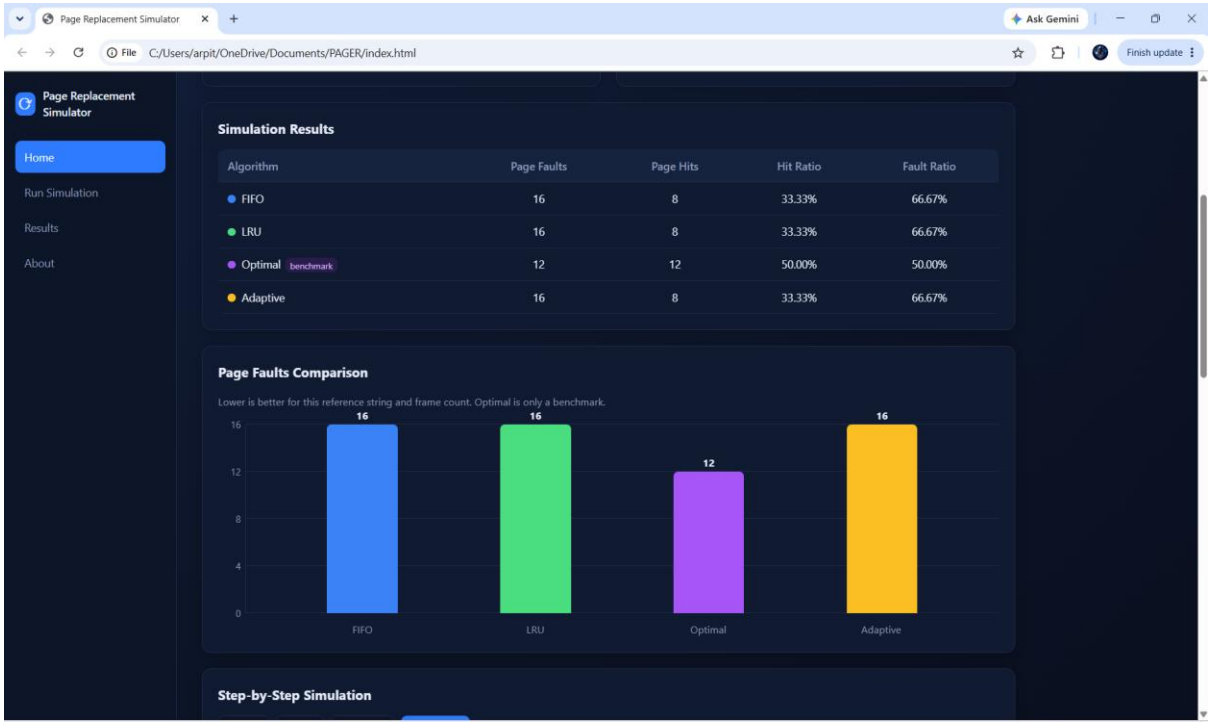
Repeated Reference:



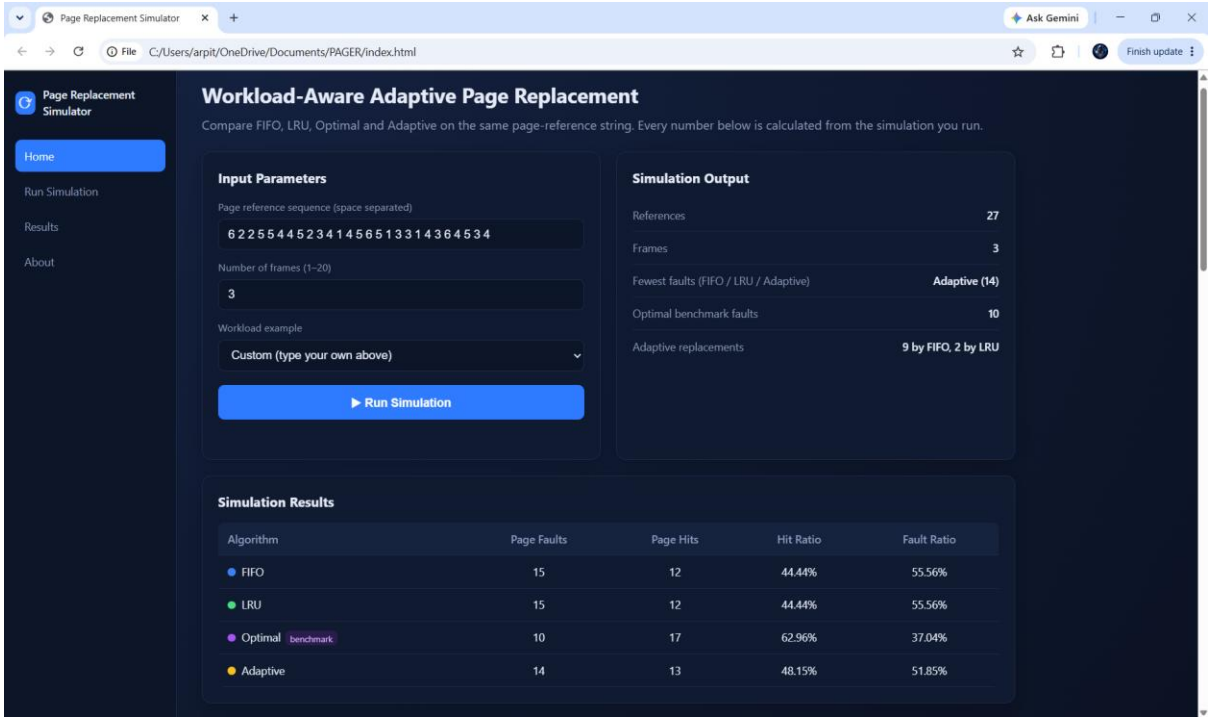


Mixed Reference:





Custom Reference:



8. GitHub Repository and Submission

GitHub Repository Link: [https://github.com/arpitraj702-del/Workload-Aware-Adaptive-Page-Replacement.git]

9. Conclusion

The Workload-Aware Adaptive Page Replacement project demonstrates the operation of common page replacement algorithms and compares their performance using page faults, page hits, and related metrics. The web interface provides a visual and interactive way to understand memory replacement decisions. The Adaptive strategy illustrates how recent workload behaviour can guide the selection between FIFO and LRU, while Optimal serves as a theoretical benchmark for comparison.

The project combines operating system concepts with C++ programming and web technologies to provide an educational tool for understanding virtual memory management.